Laboratório de Redes com Arduino

Camada Física

Redes de Computadores

Profa. Kalinka Regina Lucas Jaquie Castelo Branco

6 de abril de 2026

Sistemas Embarcados

Sistemas Embarcados:

- Poucos recursos
- Construídos para um propósito único
- Funcionam com pouca ou nenhuma interação humana

Arduino:

- Plataforma fácil de ser programada
- Placa UNO:
  - 14 pinos E/S digitais
  - 6 entradas analógicas

Sistemas Embarcados

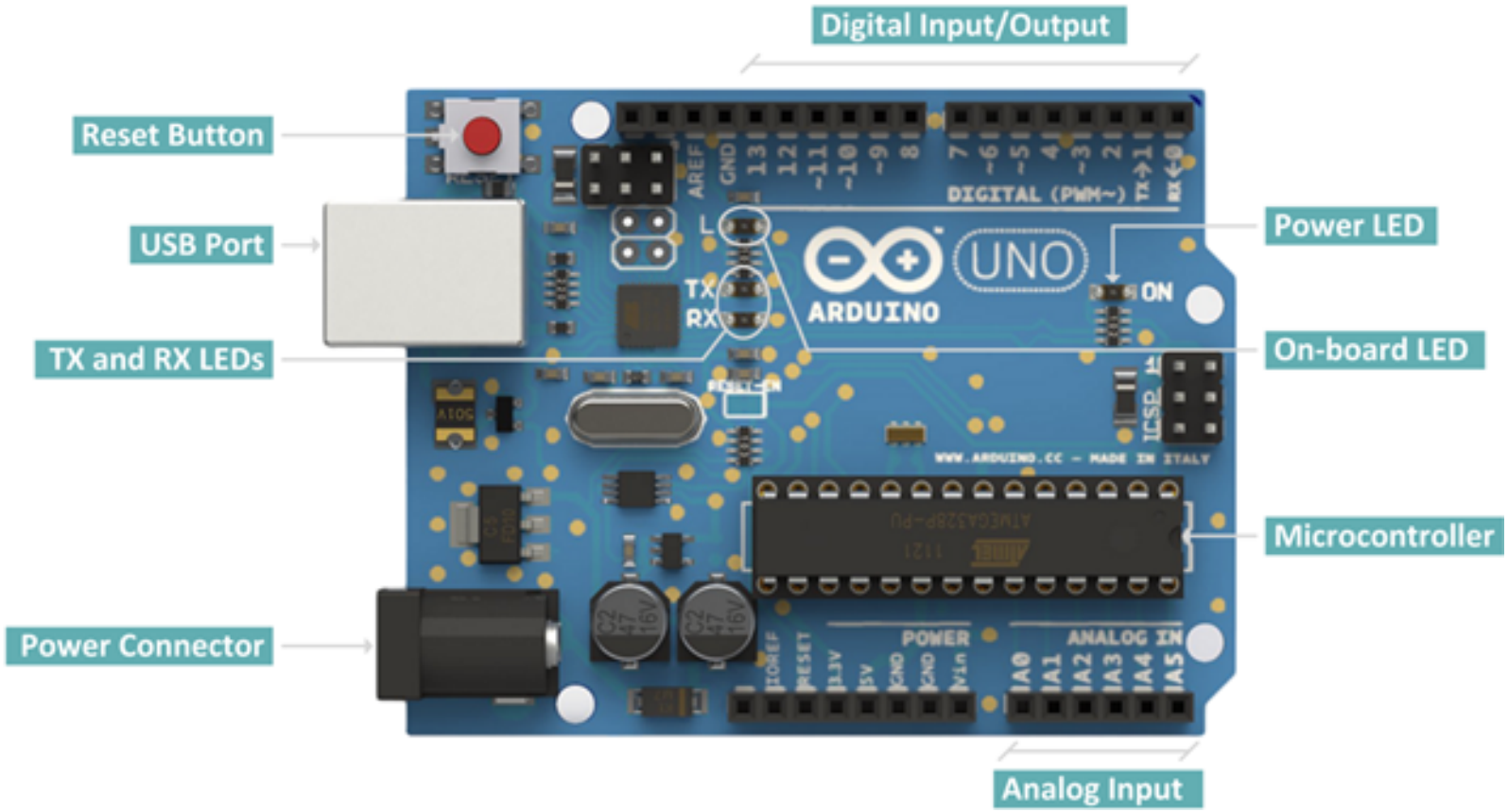
Sistemas Embarcados:

- Poucos recursos
- Construídos para um propósito único
- Funcionam com pouca ou nenhuma interação humana

Arduino:

- Plataforma fácil de ser programada
- Placa UNO:
  - 14 pinos E/S digitais
  - 6 entradas analógicas
  - ABSURDOS 32Kb de memória flash
  - ABSURDOS 2Kb de memória SRAM

Arduino Uno


Digital Input/Output
Reset Button
USB Port
TX and RX LEDs
Power Connector
Power LED
On-board LED
Microcontroller
Analog Input
DIGITAL (PWM~)
UNO
ARDUINO
ON
TX
RX
L
POWER
ANALOG IN

Atividade Prática

Objetivo

Fazer dois dispositivos Arduino conversarem por um protocolo serial assíncrono baseado no protocolo RS-232-C

Protocolo RS-232

O protocolo RS-232 é um protocolo assíncrono muito utilizado para a transmissão de caracteres (ex, teclado).

- Serial — informação em um único fio
- Assíncrono — não há informação de início ou fim do bit
- As transmissões são feitas sob demanda
- 1 bit de início (*start bit*), bits de dados, bit de paridade (opcional, par ou ímpar) e bit de parada (*stop bit* — 1, 1.5 ou 2 bits)

Protocolo RS-232 — Paridade

A paridade se refere a quantidade de bits ‘1’ transmitidos naquele dado:

- Se a paridade é **par**, deve haver um número par de ‘1’s na transmissão
- Se a paridade é **ímpar**, deve haver um número ímpar de ‘1’s na transmissão

O bit de paridade seve para alcançar a paridade desejada.

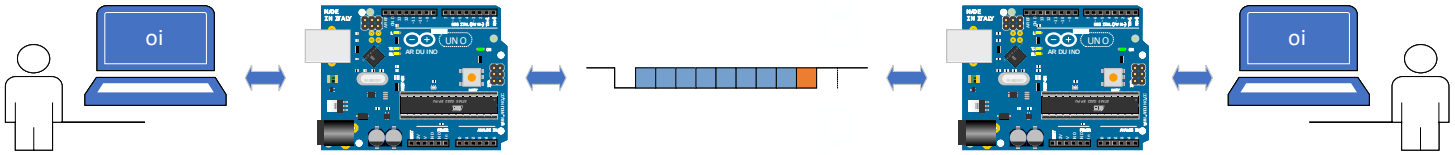
Protocolo RS-232 — Exemplo

Exemplo de transmissão: string "oi" com 8 bits de dados, paridade ímpar e 1 bit de término.

- 1º caractere: 'o' (0x6F ou 0110 1111b)
- 2º caractere: 'i' (0x69 ou 0110 1001b)

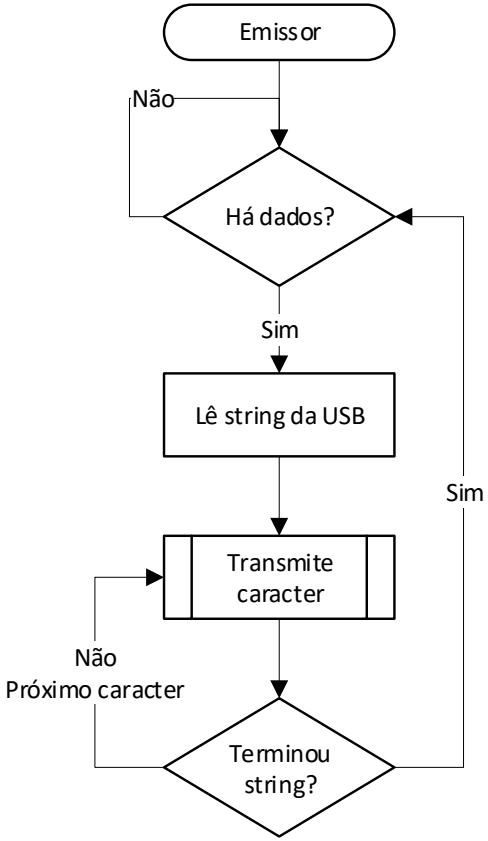
Nossa Atividade

Faremos um protocolo baseado no RS-232C para envio de caracteres no formato ASCII[1].

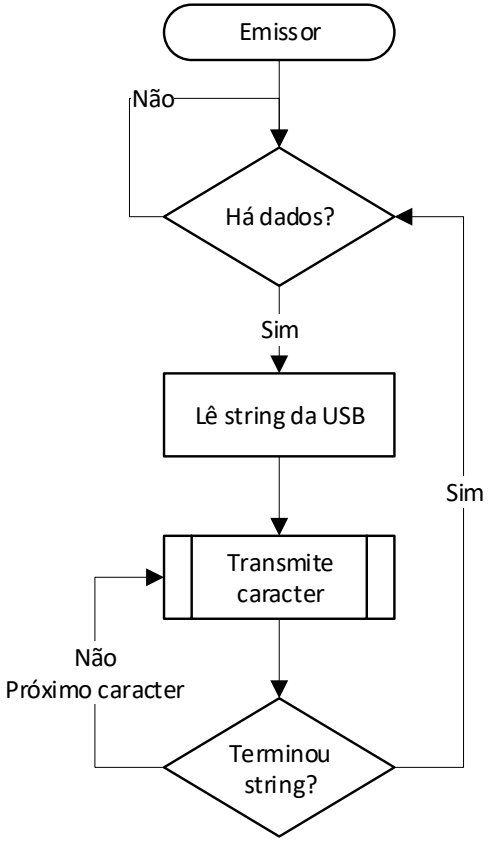


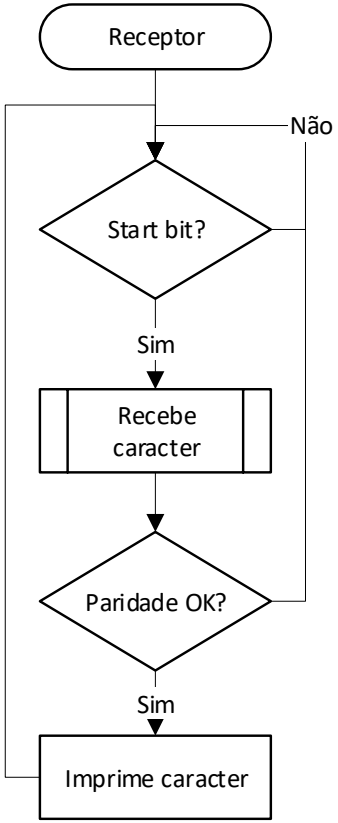
[1] https://upload.wikimedia.org/wikipedia/commons/d/dd/ASCII-Table.svg

Nossa Atividade



Nossa Atividade





Nossa Atividade

Para fazer o baud rate correto, iremos usar o **TIMER1**.

- Um temporizador ou *timer* é uma estrutura no μC que permite a contagem de tempo a partir de um *clock* interno ou externo.
- Quando o tempo programado no temporizador é atingido, ele gera uma **interrupção** no μC.
- O μC então interrompe a execução do programa e executa a **rotina de interrupção** associada àquele evento.

Nossa Atividade

Uso do temporizador:

- Incluir arquivo `Temporizador.h` no programa
- Métodos disponíveis:
  - `void configuraTemporizador(int baud_rate)`
  - `void iniciaTemporizador()`
  - `void paraTemporizador()`

Rotina de interrupção: `ISR(TIMER1_COMPA_vect){}`

Nossa Atividade

Cada grupo receberá dois parâmetros:

- emissor ou receptor
- paridade par ou paridade ímpar

- Os pinos TX do emissor e RX do receptor deverão ser o pino 13 (LED!)
- Baud rate inicial = 1Hz

Nossa Atividade

Algumas questões:

- Quando o temporizador deve ser iniciado/parado?
- Como o receptor vai saber que a transmissão começou?
- Quando o receptor deve ler o valor do bit?
- Qual o intervalo de envio entre caracteres?

Funções úteis[2]

Porta serial (Serial monitor):

```
Serial.print(``Text'')
Serial.println(``Text'')
Serial.peek()
Serial.read()
Serial.write()
```

I/O Digital:

```
pinMode(pin,[INPUT,OUTPUT])
digitalRead(pin)
digitalWrite(pin, value)
```

Tempo:

```
delay(ms)
```

Variáveis:

Modificador `volatile`

Operações de bit:

```
bitClear(x,n)
bitRead(x,n)
bitSet(x,n)
bitWrite(x,n,b)
```

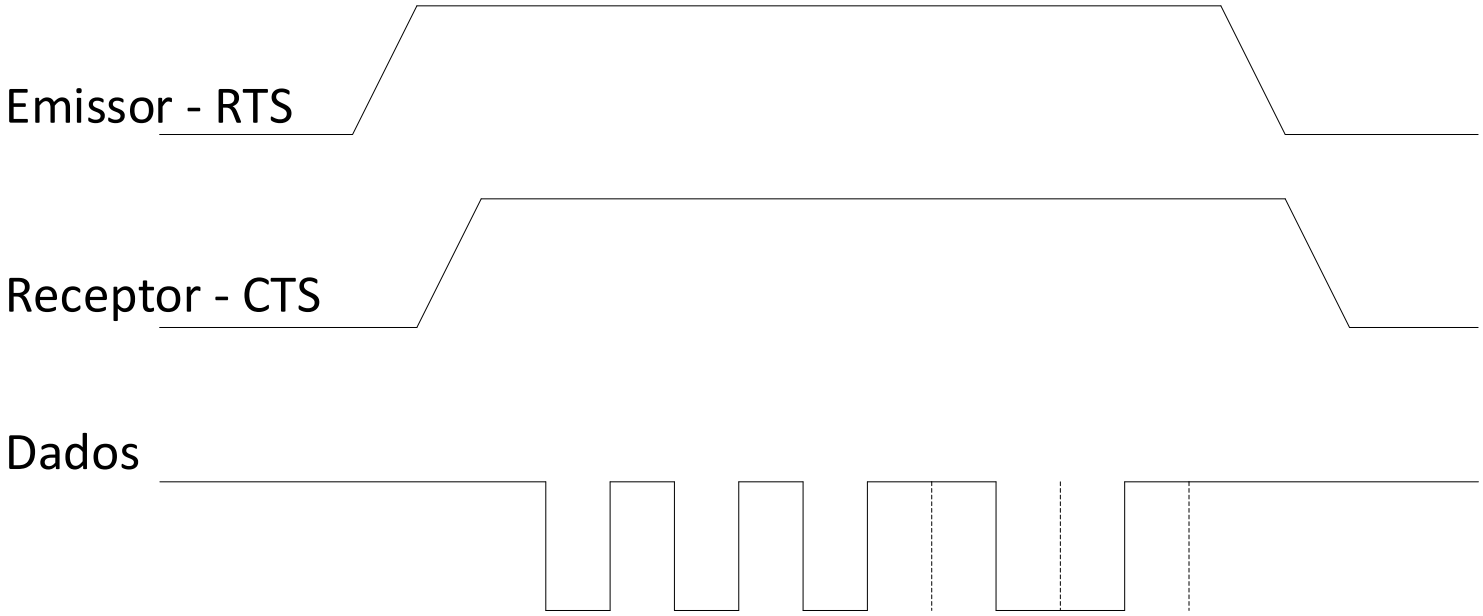
[2] Referência em `https://www.arduino.cc/reference/en/`

Esqueleto do Código

```
#include "Temporizador.h"

// Calcula bit de paridade - Par ou impar
bool bitParidade(char dado){
  //>>>> Codigo Aqui <<<<
}

ISR(TIMER1_COMPA_vect){
  //>>>> Codigo Aqui <<<<
}

void setup(){
  //>>>> Codigo Aqui <<<<
}

void loop ( ) {
  //>>>> Codigo Aqui <<<<
}
```

Desafio — *Handshake*

O *handshake* é útil para controlar o fluxo de dados e não sobrecarregar o receptor, que pode não estar disponível para a comunicação.



Vamos lá!

Bora Trabalhar!